

JudgeMesh 分布式在线判题系统软件需求说明书

案卷号：JudgeMesh-SRS-2026-01

日期：2026-05-06

项目名称：JudgeMesh 分布式在线判题系统

文档类型：软件需求说明书

作者：贾呈煜

完成日期：2026-05-06

签收人：待定

签收日期：待定

修改情况记录

版本号	修改批准人	修改人	修改日期	修改内容	签收人
V1.0	待定	贾呈煜	2026-05-06	根据 baseline《软件需求说明书编写规范》与 docs 现有材料形成正式版	待定

1 引言

1.1 编写目的

本文档用于明确 JudgeMesh 分布式在线判题系统的软件需求，作为概要设计、详细设计、编码实现、测试验收和课程答辩的共同依据。文档重点说明系统的业务范围、用户角色、功能需求、外部接口、性能指标、设计约束和质量属性，避免开发过程中出现范围蔓延、接口理解不一致或验收口径不清。

本文档的预期读者包括：

读者	关注点
任课教师与答辩评审	系统选题价值、功能边界、分布式课程主题落地、可验收指标
项目组成员	需求范围、模块职责、接口边界、开发优先级
测试人员	功能验收点、性能指标、异常场景和测试依据
运维与部署人员	运行环境、容量目标、安全约束和可用性要求

1.2 范围

待开发软件系统名称为 JudgeMesh 分布式在线判题系统。系统面向算法课程、程序设计竞赛训练和课程答辩演示场景，提供题目浏览、题目管理、代码提交、多语言沙箱判题、提交记录、比赛、实时排行榜、可观测大盘和混沌演示等能力。

系统将完成以下工作：

- 为学生提供注册登录、题目浏览、代码提交、提交结果查看、比赛参与和排行榜查看能力。
- 为出题人提供题目创建、题面维护、测试用例上传、题目发布和比赛题目配置能力。
- 为管理员提供用户管理、比赛管理、系统监控、限流熔断配置和混沌演示入口。
- 使用微服务、消息队列、分布式选主、容器编排、服务治理、可观测性和沙箱隔离支撑高并发判题场景。

1.3 定义

术语	定义
OJ	Online Judge，在线判题系统
JudgeMesh	本项目名称，表示由多个微服务和 Worker 节点协同组成的判题系统
Submission	一次代码提交记录
JudgeTask	提交服务投递给判题调度器的异步判题任务
JudgeResult	判题 Worker 回写给提交服务的判题结果
Dispatcher	判题调度器，负责消费任务、选择 Worker、处理超时和重试
Worker	判题执行节点，负责下载用例、编译代码、运行沙箱和比对输出

术语	定义
isolate	Linux 下的程序运行沙箱工具，提供 cgroups、namespaces 等隔离能力
AC	Accepted，答案正确
WA	Wrong Answer，答案错误
TLE	Time Limit Exceeded，超出时间限制
MLE	Memory Limit Exceeded，超出内存限制
RE	Runtime Error，运行时错误
CE	Compile Error，编译错误
SE	System Error，系统错误
RBAC	Role-Based Access Control，基于角色的访问控制
JWT	JSON Web Token，用于无状态身份认证
SLO	Service Level Objective，服务等级目标
HPA	Horizontal Pod Autoscaler，Kubernetes 水平自动伸缩
PACELC	分布式系统在分区和正常状态下的一致性、可用性、延迟权衡模型

1.4 参考资料

资料名称	来源
《软件需求说明书编写规范》	docs/template/软件需求说明书编写规范.doc
《概要设计说明书编写规范》	docs/template/概要设计说明书编写规范.doc
JudgeMesh 设计文档导航	docs/design/README.md
JudgeMesh 总体架构	docs/design/02-总体架构.md
JudgeMesh 服务拆分与接口	docs/design/03-服务拆分.md
JudgeMesh 数据模型与存储	docs/design/04-数据模型.md
JudgeMesh 判题流水线	docs/design/05-判题流水线.md
JudgeMesh 服务治理与容错	docs/design/06-服务治理.md
JudgeMesh 可观测性方案	docs/design/07-可观测性.md
JudgeMesh 部署架构	docs/design/08-部署架构.md
JudgeMesh 测试策略	docs/design/10-测试策略.md
JudgeMesh 四周排期	docs/design/11-四周排期.md
JudgeMesh 五人分工	docs/design/12-五人分工.md

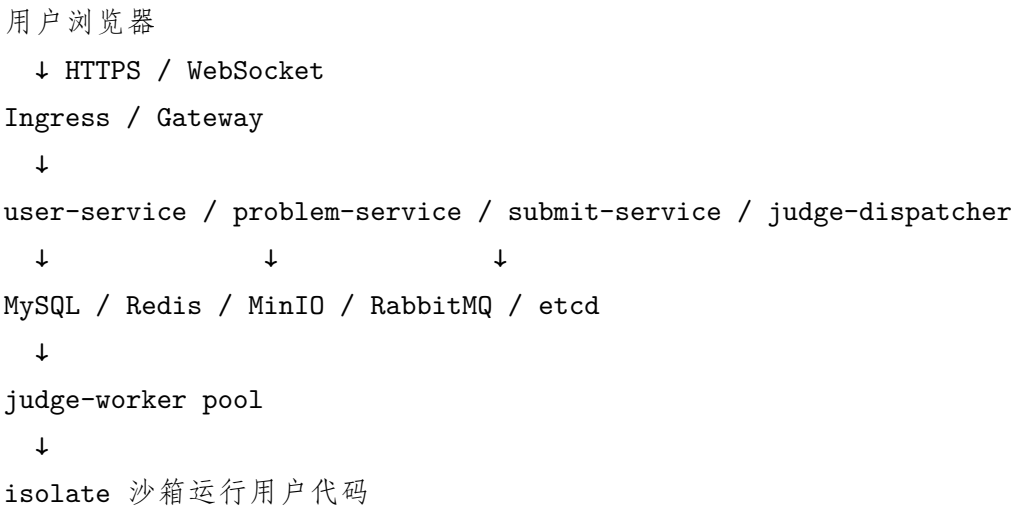
2 项目概述

2.1 产品描述

在线判题系统是算法课程和程序设计竞赛的基础设施。典型 OJ 在比赛开始、课程作业截止前和集中训练时会出现短时间大量提交，系统需要同时处理用户请求、提交入库、异步调度、沙箱执行、结果回写和排行榜更新。本项目以 OJ 为业务载体，落地课程中涉及的微服务架构、服务治理、CAP/PACELC、分布式事务、分布式共识、容器编排、可观测性和混沌工程等内容。

JudgeMesh 是一个独立的在线判题平台，同时也是一个分布式系统课程项目。系统由浏览器前端、Spring Cloud Gateway、多个业务微服务、RabbitMQ、Redis、MySQL、MinIO、etcd、judge-dispatcher、judge-worker 池、Kubernetes 和可观测性组件共同组成。

系统上下文如下：



2.2 产品功能

系统功能按优先级分为 MVP 必做能力和答辩亮点能力。

2.2.1 MVP 功能

编号	功能	简述	优先级
F-01	用户注册与登录	邮箱、用户名、密码注册，JWT 登录，Refresh Token 续期	P0

编号	功能	简述	优先级
F-02	角色权限控制	Guest、Student、Setter、Admin 四类角色，按角色授权	P0
F-03	题目浏览	题目列表、分页、搜索、标签筛选、题目详情	P0
F-04	题目管理	出题人创建、编辑、发布题目，维护时限、内存、标签和题面	P0
F-05	测试用例管理	上传测试用例文件，生成 manifest，供 Worker 下载	P0
F-06	代码提交	学生选择题目、语言和代码，系统异步判题并返回提交 ID	P0
F-07	多语言判题	支持 C、C++、Java、Python	P0
F-08	判题状态机	支持 Pending、Judging、AC、WA、TLE、MLE、RE、CE、SE	P0
F-09	提交记录	用户查看个人提交历史、状态、耗时、内存和错误信息	P0
F-10	全站排行榜	按 AC 数量和累计耗时排名	P1

2.2.2 亮点功能

编号	功能	简述	优先级
F-11	比赛模块	创建比赛、报名、比赛题目、限时提交、比赛榜单	P0
F-12	实时排行榜推送	Redis ZSet 维护排名，WebSocket 推送榜单变化	P0

编号	功能	简述	优先级
F-13	封榜机制	比赛结束前指定时间停止公开推送排名变化	P1
F-14	可观测性大盘	Grafana 展示 QPS、判题延迟、Worker 池、队列堆积、链路追踪	P0
F-15	混沌演示	管理员触发杀 Worker、杀 dispatcher leader、MQ 故障等实验	P0
F-16	调度器选主	judge-dispatcher 多副本通过 etcd Lease 选出 leader	P1
F-17	分布式事务演示	提交扣积分场景用 Seata AT 展示跨服务强一致	P1

2.3 用户特点

用户角色	用户特点	使用频度	核心权限
Guest 访客	未登录用户，对系统无管理权限	低到中	浏览公开题目和公开排行榜
Student 学生	具备基础编程能力，主要关注提交和结果	高	提交代码、查看个人记录、参加比赛
Setter 出题人	熟悉题目配置和测试用例，负责题目质量	中	题目 CRUD、上传测试用例、配置比赛题目
Admin 管理员	具备系统管理和运维意识	低到中	用户管理、比赛管理、系统监控、混沌演示
运维人员	熟悉 Kubernetes、日志、监控和发布流程	中	部署、回滚、观测、告警和容量调整

系统预期使用场景包括：

- 平时训练：学生分散提交，提交 QPS 较低，题目浏览和个人记录查询较多。
- 比赛场景：短时间内大量提交和频繁刷新排行榜，提交链路和排行榜链路压力上升。

- 答辩演示：集中展示提交到判题、排行榜推送、可观测大盘和混沌恢复。

2.4 一般约束

约束类别	约束内容
开发周期	课程项目交付周期为 4 周，第 10 周需要交付需求文档和概要设计文档
团队规模	5 人协作，按服务和目录划分职责
编程语言	Java 服务使用 JDK 17，judge-worker 使用 Go 1.22，前端使用 TypeScript
技术底座	Spring Boot 3.5、Spring Cloud 2025、Spring Cloud Alibaba 2025
部署环境	Kubernetes 1.28，学生云服务器资源有限，需控制资源消耗
通信协议	外部 HTTP/HTTPS 和 WebSocket，内部 REST、OpenFeign、RabbitMQ 消息、Redis Pub/Sub
数据库	本期 MySQL、Redis、MinIO、RabbitMQ 单实例为主，etcd 采用 3 节点共识
安全	用户代码必须在沙箱中运行，禁止未经授权访问和越权操作
可观测性	所有核心服务需暴露指标，关键链路需可追踪，错误需可定位

2.5 假设和依据

编号	假设或依据	对需求的影响
A-01	项目以课程演示和小规模真实使用为目标，不承担公网生产级 SLA	MySQL、Redis、RabbitMQ 本期可采用单实例并配备备份
A-02	服务器可提供 Linux、containerd、Kubernetes 和 privileged 容器能力	isolate 可部署在 judge-worker Pod 内
A-03	用户提交的代码长度不超过 64 KB	降低存储压力和恶意提交风险

编号	假设或依据	对需求的影响
A-04	题目测试用例由出题人提前上传并校验	判题时无需动态生成用例
A-05	本期不做 SPJ 和交互题	输出比对采用严格比对加末尾空白规整
A-06	比赛规模以课程演示和班级范围为主	目标容量定义为常态 100 QPS、比赛提交峰值 500 QPS
A-07	所有服务遵循共享 API 模块中的 DTO 和消息契约	降低多成员并行开发接口不一致风险

3 具体需求

3.1 功能需求

3.1.1 F-01 用户注册与登录 引言：用户系统提供身份认证能力，是提交、比赛、题目管理和权限控制的基础。

输入：

输入项	来源	约束
username	前端注册表单	3 到 64 字符，唯一
email	前端注册表单	合法邮箱格式，唯一
password	前端注册表单	至少 8 位，服务端 BCrypt 哈希存储
login identifier	前端登录表单	支持邮箱或用户名

加工：

- 1. 注册时校验用户名、邮箱和密码强度。
- 2. 查询用户库，确保用户名和邮箱未被占用。
- 3. 使用 BCrypt 生成密码哈希。
- 4. 创建用户记录，默认授予 Student 角色。
- 5. 登录时校验密码，成功后签发 Access Token 和 Refresh Token。
- 6. Gateway 后续解析 JWT，并向下游透传 X-User-Id 和 X-User-Roles。

输出：

输出项	说明
accessToken	1 小时有效，用于访问受保护接口
refreshToken	7 天有效，用于刷新 access token
user profile	用户 ID、用户名、昵称、角色等非敏感信息
error code	用户名重复、邮箱重复、密码错误、Token 过期等错误

3.1.2 F-02 角色权限控制 引言：系统采用 RBAC 控制敏感操作，避免学生访问题目管理、系统管理和混沌操作。

输入：

- JWT 中的 roles 声明。
- Gateway 注入的 X-User-Roles 请求头。
- 接口资源的权限配置。

加工：

1. Gateway 校验 JWT 签名和过期时间。
2. Gateway 对白名单接口放行，对非白名单接口要求登录。
3. 下游服务根据 Header 和接口注解判断用户是否具备访问权限。
4. 管理员接口、题目管理接口和混沌演示接口执行角色校验。

输出：

- 鉴权成功时转发请求。
- 未登录返回 401。
- 权限不足返回 403。

3.1.3 F-03 题目浏览 引言：题目浏览是学生日常训练的入口，要求读多写少、响应快速。

输入：

输入项	约束
page、size	分页参数，size 默认 20
keyword	按标题和题号搜索
tag	标签筛选
difficulty	难度筛选

加工：

1. Gateway 路由到 problem-service。
2. problem-service 优先读取 Redis 热门题目缓存或题目详情缓存。
3. 缓存未命中时查询 MySQL，并写回 Redis。
4. 仅返回已发布题目给普通用户。

输出：

- 题目列表：题号、标题、难度、标签、AC 数、提交数。
- 题目详情：题面 Markdown、输入输出格式、样例、时间限制、内存限制。

3.1.4 F-04 题目管理与测试用例管理 引言：出题人负责维护题目元数据和测试用例。测试用例是判题的核心输入，必须具备私有访问和 manifest 管理能力。

输入：

输入项	来源	约束
title	Setter 表单	必填，最大 255 字符
description	Setter 表单	Markdown
timeLimitMs	Setter 表单	正整数，默认 1000 ms
memoryLimitMb	Setter 表单	正整数，默认 256 MB
tags	Setter 表单	可多选
testcase files	文件上传	成对的 .in 与 .ans 文件

加工：

1. 校验 Setter 或 Admin 权限。
2. 写入 problem、problem_tag、testcase_manifest。

3. 测试用例文件上传到 MinIO 私有桶 `testcases/`。
4. 生成 manifest，供 Worker 通过预签名 URL 拉取。
5. 修改题目时失效 Redis 题目缓存。

输出：

- 创建或更新后的题目 ID。
- 测试用例 manifest。
- 上传失败、文件缺失、权限不足等错误信息。

3.1.5 F-05 代码提交 引言：代码提交是系统主链路，需要同步阶段低延迟、异步阶段可靠投递。

输入：

输入项	来源	约束
problemId	前端	必须存在且已发布
contestId	前端	可为空；若不为空需校验比赛报名和时间
language	前端	C、CPP、JAVA、PYTHON
code	前端	不超过 64 KB

加工：

1. Gateway 校验 JWT 和提交接口限流。
2. submit-service 校验题目、比赛、用户权限和防重提交。
3. 若开启积分演示，submit-service 通过 Seata 调用 user-service 扣 1 积分并写提交记录。
4. submission 初始状态设为 PENDING。
5. submit-service 投递 JudgeTask 到 RabbitMQ submit.queue。
6. 返回提交 ID，前端通过 WebSocket 或轮询查看结果。

输出：

- submissionId。
- 初始状态 PENDING。
- 输入非法、未登录、限流、题目不存在、比赛不可提交等错误。

3.1.6 F-06 多语言判题与状态机 引言：判题 Worker 负责在 isolate 沙箱内编译和运行用户代码，并将结果回写给 submit-service。

输入：

- RabbitMQ 中的 JudgeTask。
- problem-service 提供的测试用例 manifest 和预签名 URL。
- 题目时间限制、内存限制、语言和代码。

加工：

1. judge-dispatcher leader 消费任务。
2. dispatcher 按健康状态和负载选择 Worker。
3. Worker 下载用例，初始化 isolate box。
4. 编译型语言先编译，失败返回 CE。
5. 按用例运行程序，解析 isolate meta。
6. 根据输出和资源限制映射 AC、WA、TLE、MLE、RE、SE。
7. Worker 回写 JudgeResult。
8. submit-service 更新 MySQL、Redis 排行榜和 WebSocket 推送。

输出：

输出项	说明
status	AC、WA、TLE、MLE、RE、CE、SE
score	得分，MVP 可按全对 100、未全对按用例累计
timeUsedMs	最大或总耗时，单位 ms
memoryUsedKb	峰值内存，单位 KB
judgeMessage	编译错误、运行错误、系统错误等摘要
judgedByWorker	Worker Pod 名或节点标识

3.1.7 F-07 提交记录 引言：用户需要查看自己的历史提交，出题人和管理员需要查看题目维度或全站维度的提交记录。

输入：

- submissionId。
- 当前用户 ID 和角色。
- 分页、状态、语言、题目、比赛等筛选条件。

加工：

1. 普通学生只能查看自己的提交详情。
2. Setter 可查看自己题目的提交统计，不默认暴露他人完整代码。
3. Admin 可查看全量提交和系统错误。
4. 查询按 user_id + submitted_at、problem_id + status、contest_id + user_id 索引优化。

输出：

- 提交列表：题目、语言、状态、得分、耗时、内存、提交时间。
- 提交详情：代码、错误信息、判题节点、traceId。

3.1.8 F-08 比赛模块 引言：比赛模块体现高并发和实时排行榜场景，是答辩重点之一。

输入：

输入项	说明
contest title	比赛名称
startTime、endTime	比赛起止时间
freezeBeforeMin	封榜提前分钟数，默认 30
problemIds	比赛题目集合
registeredUserIds	报名用户集合

加工：

1. Admin 或 Setter 创建比赛并绑定题目。
2. 学生在比赛开始前或规则允许时间内报名。
3. 提交时校验当前时间、报名状态和题目归属。
4. AC 后更新 Redis ZSet rank:contest:{contestId}。

5. 封榜时间到达后，服务端停止公开推送排名变化。

6. 比赛结束后管理员解榜或展示最终榜。

输出：

- 比赛列表和详情。
- 报名结果。
- 比赛排行榜。
- 封榜状态。

3.1.9 F-09 实时排行榜推送 引言：排行榜要求读写频繁、响应快速，采用 Redis ZSet 和 WebSocket。

输入：

- JudgeResult 中的 AC 状态。
- 比赛 ID、用户 ID、提交时间和错误提交次数。
- WebSocket 订阅请求。

加工：

1. submit-service 在首次 AC 时更新全站榜和比赛榜。
2. ZSet 分数按 AC 数和罚时计算。
3. WebSocket 客户端首次连接拉取全量榜单。
4. 后续通过 Redis Pub/Sub 推送 delta。
5. 封榜期间过滤公开排名推送。

输出：

- 全站排行榜。
- 比赛排行榜。
- 排名变化 delta。

3.1.10 F-10 可观测性与混沌演示 引言：系统需展示分布式运行状态，并验证单点故障下的恢复能力。

输入：

- 应用指标、Kubernetes 指标、中间件指标、日志和链路追踪。
- 管理员触发的混沌实验命令。

加工：

1. Spring 服务暴露 /actuator/prometheus。
2. Go Worker 暴露 /metrics。
3. Prometheus 抓取指标，Grafana 展示大盘。
4. SkyWalking 展示提交链路。
5. Loki 聚合 JSON 日志。
6. ChaosMesh 注入 PodKill、NetworkDelay、CPUSTress 等故障。

输出：

- 业务总览、判题流水线、服务治理、基础设施 4 类大盘。
- 告警事件。
- 混沌实验结果和恢复时间。

3.2 外部接口需求

3.2.1 用户接口 系统通过 Web 前端提供用户界面，前端采用 React、Vite 和 TypeScript 实现。

页面或区域	用户	主要功能
登录注册页	Guest	注册、登录、错误提示
首页	全部	热门题目、系统状态、入口导航
题目列表页	全部	分页、搜索、标签筛选、难度筛选
题目详情页	Student、Setter、Admin	查看题面、选择语言、提交代码
提交记录页	Student	查看个人提交和判题详情
比赛列表页	Student	浏览比赛、报名比赛
比赛详情页	Student	查看比赛题目、提交和榜单

页面或区域	用户	主要功能
排行榜页	全部	查看全站榜和比赛榜
题目管理页	Setter、Admin	创建、编辑、发布题目和上传用例
管理后台	Admin	用户管理、比赛管理、混沌演示、系统状态

用户接口要求：

- 输入表单必须进行前端基础校验和服务端最终校验。
- 判题结果需要支持 WebSocket 实时更新，同时提供轮询兜底。
- 编译错误和运行错误需要展示裁剪后的错误信息，避免过长文本影响页面。
- 重要操作需要有明确的成功或失败反馈。

3.2.2 硬件接口 系统不直接对接专用硬件，主要依赖通用服务器资源和 Linux 容器运行环境。

硬件或资源	要求
CPU	Worker 节点需要支持多核并发判题
内存	单 Worker Pod 推荐 2 GiB limit
磁盘	MySQL、Redis、RabbitMQ、MinIO、etcd 需使用持久化卷
网络	Kubernetes 集群内服务互通，Ingress 提供外部入口
Linux 内核能力	isolate 需要 cgroups、namespaces，Pod 需具备相应权限

3.2.3 软件接口

软件或服务	接口方式	用途
MySQL 8	JDBC / MyBatis-Plus	用户、题目、提交、比赛等持久化
Redis 7	Spring Data Redis	缓存、排行榜、Pub/Sub、防重提交
RabbitMQ	Spring AMQP	判题任务队列、重试队列、死信队列
MinIO	S3 API / SDK	测试用例、头像、备份文件
etcd 3.5	jetcd	dispatcher 选主、inflight 任务记录
Nacos	Spring Cloud Alibaba	服务注册和配置中心
Sentinel	Spring Cloud Alibaba	限流、熔断、降级
Seata	Seata AT	提交扣积分演示事务
Prometheus	HTTP scrape	指标采集

软件或服务	接口方式	用途
SkyWalking	Agent / OAP	链路追踪
Loki	Promtail	日志聚合
Kubernetes	API Server	部署、服务发现、HPA、Pod 管理

3.2.4 通信接口

通信方向	协议	说明
Browser 到 Gateway	HTTPS	REST API 入口
Browser 到 Gateway	WebSocket over HTTPS	提交状态和排行榜推送
Gateway 到业务服务	HTTP + Spring LoadBalancer	按服务名路由
服务间同步调用	OpenFeign / HTTP	查询用户、题目等同步数据
submit-service 到 dispatcher	RabbitMQ	异步投递判题任务
dispatcher 到 worker	HTTP JSON	派发具体判题任务
worker 到 submit-service	HTTP JSON	回写判题结果
服务到 Redis	TCP	缓存、ZSet、Pub/Sub
服务到 MySQL	TCP	事务和持久化

3.3 性能需求

指标	目标值	测量方式
提交接口同步阶段 P99	小于 200 ms	wrk、k6、JMeter
判题端到端 P50	小于 2 s，基于 C++ 简单题	业务埋点
判题端到端 P99	小于 8 s	业务埋点
常态提交吞吐	大于 100 QPS	压测
比赛提交峰值	大于 500 QPS，同步入队不崩溃	压测
排行榜查询 P99	小于 50 ms	wrk、Redis 指标
题目详情 P99	小于 100 ms，Redis 命中率大于 90%	压测
Worker 单节点能力	简单题约 2 提交/s	Worker 指标
dispatcher 选主恢复	leader 宕机后 10 s 内恢复	ChaosMesh
Worker 宕机恢复	任务 30 s 内重新派发	ChaosMesh

容量目标：

- 平时平均 5 提交/s。
- 比赛瞬时 50 提交/s，队列削峰后持续处理约 20 提交/s。
- Worker Pod 池默认 3 副本，HPA 最大扩容至 20 副本。

3.4 设计约束

3.4.1 其他标准的约束

- 文档结构遵循 baseline 中的软件需求说明书和概要设计说明书编写规范。
- 代码提交、分支、PR 和模块责任遵循仓库协作规范。
- Java 服务统一使用顶层 Maven BOM，禁止子模块覆盖核心框架版本。
- 数据库变更通过 Flyway 管理。
- 接口和消息结构必须优先在 services/api 共享模块定义。
- 日志必须包含 service、traceId、userId、submissionId 等关键字段，禁止打印密码和 JWT。

3.4.2 硬件的限制

- 学生云服务器资源有限，演示环境可能只有 1 master + 2 到 3 worker 节点。
- 中间件单实例部署时可用性有限，需通过备份和告警缓解。
- isolate 需要较高容器权限，本期在课程集群中接受 privileged 风险，并在答辩中说明。
- HPA 最大副本数受云资源限制，演示时可将 maxReplicas 调低到 8。

3.5 属性

3.5.1 可用性

场景	要求
单 Worker 宕机	不影响系统整体可用，30 s 内任务重新派发
单业务服务实例宕机	Gateway 和 Kubernetes 自动剔除故障实例
dispatcher leader 宕机	etcd Lease 10 s 内过期并选出新 leader
所有 Worker 不可用	提交服务继续入队或网关限流，前端给出友好提示

MQ 不可用

submit-service 进入降级路径或返回 503，并触发告警

系统提交接口可用率目标为 99.5% 以上，课程演示期以压测和混沌实验数据为验收依据。

3.5.2 安全性

安全维度	需求
身份认证	JWT 验签，Access Token 1 小时有效，Refresh Token 7 天有效
权限控制	RBAC，敏感接口要求 Setter 或 Admin
密码安全	BCrypt 哈希存储，不记录明文密码
SQL 注入	使用 MyBatis-Plus 参数化查询
XSS	前端渲染 Markdown 需要过滤危险 HTML，设置 CSP
沙箱隔离	isolate 限制 CPU、内存、进程数、文件、网络和系统调用
限流	Gateway 按 IP、userId 和接口资源限流
敏感信息	密钥通过 K8s Secret 或环境变量注入，禁止入 Git
审计	登录、提交、管理操作和混沌操作记录日志

3.5.3 可维护性

- 服务按用户、题目、提交、调度、判题执行等领域拆分。
- 每个服务拥有自己的核心数据，不跨服务直接写库。
- DTO、Feign 接口、MQ 消息结构放在共享 API 模块。
- 配置集中在 Nacos 和 K8s ConfigMap/Secret。
- 关键业务逻辑需要单元测试和集成测试。
- 指标、日志和链路追踪从项目早期接入，便于定位问题。

3.5.4 可转移与转换性

- 本地开发可使用 Docker Compose 启动中间件。
- 演示和集成环境使用 Kubernetes 部署。
- 对象存储通过 S3 兼容接口封装，未来可从 MinIO 替换为云厂商 OSS。
- Worker Runner 设计为可插拔，未来可从 isolate 替换为 Docker、gVisor 或其他沙箱。

- RabbitMQ 可在高峰吞吐需求上升后迁移到 RocketMQ 或 Kafka，业务侧通过消息 schema 降低迁移成本。

3.5.5 警告 以下需求必须使用可验证指标验收，不能只做口头描述：

- 提交接口 P99、端到端判题 P99、排行榜查询 P99。
- Worker 宕机重新派发时间。
- dispatcher leader 宕机重新选主时间。
- 沙箱对 fork bomb、死循环、内存耗尽、非法文件访问和联网行为的拦截效果。
- WebSocket 在断线后通过拉取接口恢复数据的能力。

3.6 其他需求

3.6.1 数据库 系统数据库分为用户库、题目库和提交库，缓存和队列使用 Redis 与 RabbitMQ，测试用例使用 MinIO，调度协调使用 etcd。

数据域	存储	保留时间	访问特征
用户、角色、余额	MySQL users_db	永久	写少读多
题目、标签、用例 manifest	MySQL problems_db	永久	写极少读极多
提交、比赛、报名	MySQL submits_db	永久	写多读多
排行榜和缓存	Redis	排行榜持久，缓存按 TTL	高频读写
测试用例文件	MinIO	永久	一次上传，多次读取
判题任务	RabbitMQ	未消费消息持久化	高频写，异步消费
选主和 inflight	etcd	短期	小数据强一致

备份要求：

- MySQL 每日备份到 MinIO backups/daily/。
- Redis 排行榜可定时从 MySQL 重建，避免缓存丢失导致榜单不可恢复。
- RabbitMQ 队列和消息设置 durable 和 persistent。

3.6.2 操作

系统常规操作包括：

- 用户注册、登录、刷新 Token、登出。
- 出题人创建题目、上传用例、发布题目。
- 学生提交代码、查看判题状态、查看历史记录。
- 管理员创建比赛、管理用户、查看监控、触发混沌实验。
- 运维人员部署服务、导入 Nacos 配置、执行数据库迁移、查看日志和指标。

后援与恢复操作包括：

- Kubernetes Deployment 滚动发布和 rollout undo 回滚。
- Nacos 配置版本回滚。
- Flyway 数据库迁移记录检查。
- Redis 排行榜从 MySQL 重建。
- RabbitMQ DLQ 人工检查和重放。

3.6.3 场合适应性需求

场合	适配要求
本地开发	Docker Compose 提供 MySQL、Redis、RabbitMQ、MinIO、etcd 等依赖
课程演示	允许单实例中间件，保留核心分布式能力和可观测性
比赛压测	提高 Gateway 限流阈值和 Worker HPA 上限
资源紧张	降低 Worker 最大副本数，关闭部分非必要大盘或日志采集
安全演示	使用预制恶意代码样例验证沙箱限制

4 附录

4.1 判题状态机

```
PENDING -> JUDGING -> AC
                -> WA
                -> TLE
                -> MLE
```

- > RE
- > CE
- > SE

SE 可按重试策略重新回到 PENDING，超过 3 次后成为终态。

4.2 需求优先级定义

优先级	定义
P0	必须完成，缺失则主流程不可验收
P1	应完成，是答辩或体验的重要增强
P2	可选完成，不影响主流程验收

4.3 需求到设计文档追踪表

需求	主要设计依据
用户、权限	03-服务拆分与接口、04-数据模型与存储、06-服务治理与容错
题目和用例	03-服务拆分与接口、04-数据模型与存储
提交和判题	03-服务拆分与接口、05-判题流水线
比赛和排行榜	03-服务拆分与接口、04-数据模型与存储
调度器选主	02-总体架构、05-判题流水线
可观测性	07-可观测性方案
部署和恢复	08-部署架构、06-服务治理与容错
测试验收	10-测试策略